Introduction à la programmation

Bases du langage Python

Antoine Gauquier
antoine.gauquier@ens.fr

10/09/2024



Table des matières

1. Les opérations arithmétiques
2. Les variables
3. Les autres opérateurs
4. Les listes Python

Python en tant que calculatrice

La façon la plus simple et naïve d'utiliser Python est de le considérer comme une *calculatrice*.

Les opérations *arithmétiques* sur les nombres en Python suivent les règles de priorités opératoires usuelles, et incluent donc également l'utilisation des parenthèses.

Ainsi :

- `12*(6+4)` ne donnera pas le même résultat que `12*6+4`.
- Dans `5+9*8`, c'est `9*8` qui est calculé avant de faire au résultat `+5`.
- etc.

Opérations arithmétiques élémentaires

Comme présenté dans la diapositive précédente, Python supporte les opérations arithmétiques élémentaires :

- L'addition : $+$
- La soustraction : $-$
- La multiplication : $*$
- La division : $/$

La division euclidienne

Python supporte d'autres opérations arithmétiques un peu plus avancées :

- *La division euclidienne et son reste*

La division euclidienne (ou division entière) se fait avec `//`. On peut obtenir le *reste* de cette division en utilisant l'opérateur `%`.

Par exemple : `14//3` donne `4`, car $3 \times 4 \leq 14$ mais $3 \times 4 > 14$.

Par conséquent, `14%3` donne `2`, car $3 \times 4 + 2 = 14$.

Le calcul de puissances

- *Le calcul des puissances*

Le calcul de puissances (ou *exponentiation*) se fait en Python avec l'opérateur `**`.

Ainsi `3**3` est équivalent à 3^3 et vaut 27.

Il est également possible de l'implémenter vous-même (au même titre que l'opérateur de multiplication) : vous pourrez le faire une fois que vous maîtriserez les *boucles* ...

A vous de jouer !

Exercice

Calculer la valeur de $\frac{50-5*6+20}{4}$.

Exercice

Calculer la valeur décimale (approchée) de la fraction $\frac{17}{3}$. *Vérifier la valeur obtenue en utilisant les opérateurs de division euclidienne et de reste sur ces mêmes valeurs.*

Exercice

Quel résultat `16 / 4 * 3 / 2` *doit retourner d'après-vous ? Vérifier en exécutant le code Python. Que devez-vous écrire en Python pour obtenir le résultat de* $\frac{16}{4} \times \frac{3}{2}$ *?*

Exercice

Calculer ***de deux façons différentes*** *la valeur de* 2^5 *en Python.*

Table des matières

1 Les opérations arithmétiques

2 Les variables

3 Les autres opérateurs

4 Les listes Python

Variable : qu'est-ce que c'est ?

En Python (mais aussi dans tous les autres langages de programmation), les *variables* sont des espaces dans lesquels on peut stocker une valeur (donnée) de façon persistante.

Elles sont représentées par une *adresse mémoire*, à savoir la localisation de l'emplacement dans la mémoire vive à partir de laquelle est stockée la donnée que contient la variable.

Créer (définir) une variable (1)

En Python, il n'existe pas de commande pour créer une variable. Une variable est créée **au moment où on lui assigne une valeur pour la première fois**.

Pour assigner une valeur à une variable, on utilise un opérateur d'*assignation*, à savoir =, entre le nom de la variable (à gauche) et sa valeur (à droite).

Par exemple, `maVariable = 4` aura pour conséquence d'associer à la variable `maVariable` la valeur `4`.

Le choix du *nom* de la variable est libre : il peut comporter des lettres, chiffres ou le caractère `_`.

Créer (définir) une variable (2)

Un nom de variables doit toutefois respecter certaines contraintes :

- Il ne peut commencer par un chiffre.
- Il ne peut pas contenir de mots réservés par le langage lui-même, qu'on appelle *mots-clés* du langage. [1]
- Il est **fortement déconseillé** d'utiliser des accents ou des caractères spéciaux, par soucis de compatibilité avec des environnements différents.

Au delà de ces restrictions, une bonne pratique (fondamentale) est de donnée à ses variables des **noms qui font sens** ! Cela permet non seulement de comprendre vous-même votre code si vous revenez dessus plus tard, mais aussi aux autres de le comprendre ...

1. La liste de tous les mots-clés peut être trouvée ici : `https://docs.python.org/3.11/reference/lexical_analysis.html#index-13`.

Les différents types de variables

Dans l'exemple vu précédemment (`maVariable = 4`), on a vu comment assigner un entier à une variable.

Ce n'est pas le seul *type* de donnée que l'on peut assigner à une variable (et utiliser de façon générale en Python), il en existe plusieurs autres :

- Les entiers, dont le type est `int`
- Les nombres à virgule flottante, dont le type est `float`
- Les nombres complexes, dont le type est `complex` (rarement utilisé)
- Les chaînes de caractères, dont le type est `str`
- Les variables booléennes, dont le type est `bool`
- D'autres types ignorés dans ce cours d'introduction ou vus plus loin.

Les nombres à virgule flottante (1)

Il s'agit des nombres décimaux, ou « nombres à virgule ». Leur nom provient de la façon dont ils sont représentés en mémoire, que nous ne verrons pas dans ce cours.

Il est important de garder en tête que ces nombres sont des *approximations*. Cette approximation est nécessaire pour certaines fractions qui n'ont pas de représentation exacte : $\frac{1}{3}$ par exemple.

Mais elle intervient même pour des nombres décimaux/fractions dont la représentation est exacte. Par exemple, le nombre à virgule `0.1` (représentant la fraction $\frac{1}{10}$ par exemple), vaut en réalité `0.1000000000000000055511151231257827021181583404541015625` dans sa représentation binaire en mémoire.

Les nombres à virgule flottante (2)

On remarque que l'approximation vaut la valeur exacte jusqu'à la quinzième décimale : c'est amplement suffisant pour nous !

Il est toutefois important d'avoir conscience de cette approximation, qui génère une *instabilité numérique* lorsque l'on a besoin de valeurs très précises (en optimisation mathématique par exemple ...).

Comme montré dans les exemples précédents, le séparateur (la « virgule ») est le point (.), malgré son nom qui indique le contraire.

Exemple d'assignation : `variableFlottante = 3.14`

Les chaînes de caractères

Les chaînes de caractère (ou *strings* en anglais) permettent de stocker des caractères, par exemple écrits au clavier.
Afin d'indiquer au langage de programmation qu'il s'agit d'une chaîne de caractères et non d'une portion de code, on entoure la chaîne de caractère de guillemets ou d'apostrophes.

Ainsi, si l'on veut assigner à la variable `salutation` la chaîne de caractères « Bonjour », on peut écrire `salutation = "Bonjour"` ou bien `salutation = 'Bonjour'`.

On verra plus tard dans le cours que les chaînes de caractère peuvent être vues comme des *listes Python de caractères*.

Les booléens

Les *booléens* proviennent du domaine de la logique mathématique, et plus précisément de *l'algèbre de Boole*.

Ce type peut prendre deux valeurs différentes : `True` (vrai) ou `False` (faux). Elles servent principalement à vérifier des *assertions*, et sont omniprésentes dans les tests, que vous verrez mercredi.

Ainsi, par exemple, `print(9 > 10)` vous affichera la valeur booléenne `False`, alors que `print(3.14 > 2.72)` vous affichera `True` (testez vous-même !).

Il existe un certain nombre d'opérateurs de comparaison qui, appliqués entre deux valeurs, donnent des booléens. Nous les verrons plus tard.

Quelques fonctions ...

Bien que vous verrez les *fonctions* plus tard dans la semaine, en voici quelques-unes d'utiles concernant les variables :

- La fonction `type()` : en mettant dans les parenthèses le nom d'une variable déjà définie, vous obtiendrez en retour le type de cette variable.
- Le *casting* de variables : vous pouvez *changer* ou *spécifier* le type d'une variable déjà définie. Le cas le plus courant se fait entre les `int` et les `float` : Si l'on a `maVariable = 4` (donc un entier ; `type(maVariable)` donnerait `int`), alors `float(maVariable)` changerait la valeur de la variable en `4.0`. Reproduisez l'exemple pour vous en assurer !

A vous de jouer !

Exercice

Définissez trois variables décimales `a`, `b` *et* `c` *de votre choix. Vérifiez qu'elles possèdent bien le type* `float`*. Calculez leur produit. Déterminez leur classement par ordre croissant en les comparant deux à deux avec l'opérateur de comparaison* `<`*.*

Exercice

On exécute à la suite les lignes de code suivantes : `a = 7`, `b = a - 4` *et* `print(a * b)`*. Quelle valeur sera affichée ?*

Exercice

On souhaite calculer la distance euclidienne entre les points $A = (3, 4)$ *et* $B = (6, 8)$*. Dans un premier temps, représenter chacun des points par deux variables. Ensuite, à partir des variables définies, écrire la formule permettant le calcul de la distance euclidienne entre ces deux points (pour rappel :* $d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$*). Vérifier.*

Table des matières

1 Les opérations arithmétiques

2 Les variables

3 Les autres opérateurs

4 Les listes Python

Autres familles d'opérateurs

Viennent avec les nouveaux types découverts de nouvelles familles d'opérateurs :

- Les opérateurs d'*assignation*
- Les opérateurs de *comparaison*
- Les opérateurs *logiques*
- D'autres opérateurs omis dans ce cours d'introduction ou présentés plus tard (identité, appartenance, opérations *bit-wise*, etc.).

Les opérateurs d'assignation

Nous avons déjà vu un des opérateurs de cette famille, afin d'affecter une valeur à une variable (=).

Il en existe d'autres, qui sont utiles en pratique afin de réduire la *verbosité* du code. Ils permettent de réduire :

`maVariable = maVariable OPERATEUR` *valeur*

en

`maVariable OPERATEUR=` *valeur*

où `OPERATEUR` est un des opérateurs *arithmétiques*, et *valeur* une valeur quelconque.

Les opérateurs d'assignation

Ainsi, on définit les nouveaux opérateurs d'assignation suivants :

- `+=` (réduit `x = x + 3` en `x += 3`).
- `-=` (réduit `x = x - 3` en `x -= 3`).
- `*=` (réduit `x = x * 3` en `x *= 3`).
- `/=` (réduit `x = x / 3` en `x /= 3`).
- `//=` (réduit `x = x // 3` en `x //= 3`).
- `%=` (réduit `x = x % 3` en `x % = 3`).
- `**=` (réduit `x = x ** 3` en `x **= 3`).

Les opérateurs de comparaison

Comme leur nom l'indique, les opérateurs de *comparaison* permettent de *comparer* deux valeurs entre elles. Les voici :

- `>` est l'opérateur « strictement supérieur à »
- `<` est l'opérateur « strictement inférieur à »
- `>=` est l'opérateur « supérieur ou égal à »
- `<=` est l'opérateur « inférieur ou égal à »
- `==` est l'opérateur « égal à »
- `!=` est l'opérateur « différent de »

La valeur retournée par l'utilisation d'un de ces opérateurs est un booléen. Elle indique en particulier, dans le cas des opérateurs d'ordonnancement (les 4 premiers), si l'ordre indiqué par l'opérateur est correct ou non (si on obtient `False`, on peut d'ailleurs conclure que l'assertion obtenue en inversant l'opérateur est vraie. Pour rappel : l'inverse de $>$ est $\leq$, de $<$ est $\geq$, etc).

Les opérateurs de comparaison

Ces opérateurs s'utilisent naturellement sur les types numériques (`int` ou `float`) par exemple.

Pour les autres types, on peut également vouloir définir un ordonnancement. Pour les types usuels, il vous faut comprendre le comportement donné à ces opérateurs avant de les manipuler, sans quoi le comportement de votre code ne correspondra pas nécessairement à ce que vous attendez de lui.

Pour les autres types que ce sont les types que l'on créé soit-même (que l'on appelle classes ...Vous verrez cela plus tard !), il est possible de spécifier le comportement que doivent adopter ces opérateurs. On parle alors de surcharge d'opérateur.

Les opérateurs de comparaison

Pour les chaînes de caractère par exemple, la règle retenue pour l'utilisation de l'opérateur $<$ (que l'on peut dériver aux autres opérateurs) est l'ordre lexicographique sur les caractères (l'ordre de la table ASCII), et ce de gauche à droite.

Les opérateurs de comparaison

ASCII : American Standard code for Information Exchange

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	”	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	’	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

Les opérateurs de comparaison

Concrètement, pour deux chaînes de caractères `str1` et `str2`, on va comparer deux à deux, de gauche à droite, les caractères de l'une et l'autre des chaînes.

Exemples :

- `"abc" < "acb"`
- `"adbe" > "ae"`
- `"cba" == "cba"`
- `"aaf" != "mgb"`
- `"abcdef" < "abcdez"`

Les opérateurs de comparaison

Attention à l'opérateur == : si vous l'utilisez sur des types où ce dernier n'est pas défini, Python va considérer que les deux variables comparées sont égales seulement si *leurs adresses mémoire*[2] sont égales ... ce qui en général ne nous intéresse que très peu.

2. Pour rappel, l'adresse mémoire d'une variable est la localisation dans la mémoire vive à partir de laquelle est écrite (en binaire) la valeur associée

Les opérateurs logiques

Les opérateurs *logiques* s'utilisent entre deux variables (ou valeurs) booléennes, et renvoient une autre valeur booléenne. Ils respectent par ailleurs la priorité opératoire imposée par les parenthèses.

Il en existe trois :

- `and` qui correspond à l'opération logique « et ». a `and` b renvoie `True` si et seulement si a et b valent toutes les deux `True`.
- `or` qui correspond à l'opération logique « ou ». a `or` b renvoie `True` si et seulement si au moins l'une des deux variables a ou b vaut `True`.
- `not` qui correspond à l'opération logique « non ». **Il se place devant une variable booléenne (mais rien derrière)**. `not` a renvoie `True` si a vaut `False` et vice-versa. Le `not` en Python supporte par ailleurs les lois de De Morgan.

Combinaison des opérateurs logiques et de comparaison

Comme expliqué, les opérateurs logiques et de comparaison renvoient tous des valeurs booléennes : il est donc possible (et usuel) de les combiner.

Exemples :

- `a < 10 and a >= 5` vérifie que la variable `a` se trouve dans l'intervalle $[5, 9]$ (on remarquera qu'ici `a` peut très bien être un entier ou un nombre à virgule flottante).
- `(x%3 == 0) or (x%5 == 0)` est vrai si `x` est un multiple de 3 ou de 5.

Lorsque, comme dans le premier exemple, l'on s'intéresse à une succession d'opérateurs `and`, il est possible d'écrire une version compressée des opérateurs de comparaison.
On pourrait alors réécrire `a < 10 and a >= 5` en `5 <= a < 10`. On peut utiliser ce procéder pour obtenir des expressions plus complexes, par exemple : `a < 10 < a*10 < 100`.

A vous de jouer !

Exercice

Transformer `x = ((((x + 5) * 4) ** 2) / 10)` *en 4 lignes différentes de code faisant intervenir les opérateurs d'assignation.*

Exercice

Pour le programme Python qui suit, écrire l'affichage attendu pour chaque ligne. Une fois fini, écrire le code et l'exécuter pour vérifier l'affichage obtenu.

```
a = False
b = False
x = not(a)
y = not(b)
print(a or b)
print(x or y)
print(x and y)
print(a and x)
```

A vous de jouer !

Exercice

On pose `a = 8`, `b = 3`, `c = 15` *et* `d = 6`. *Retourner le booléen obtenu en évaluant l'assertion suivante : « (a est divisible par b ou (c − a) est plus grand que 3b) et d n'est pas divisible par b et la somme de toutes les variables dépasse 30 ». Si le booléen obtenu est* `False`, *déterminer les conditions à changer pour qu'il devienne* `True`. *Vérifier ensuite en changeant celles-ci dans votre code et en la ré-exécutant.*

Table des matières

1 Les opérations arithmétiques

2 Les variables

3 Les autres opérateurs

4 Les listes Python

Les listes Python

Les listes Python permettent de stocker plusieurs éléments au sein d'une unique variable. Les éléments qui forment cette liste peuvent avoir des types différents, ce qui permets une grande flexibilité.

Celles-ci ont leur propre type Python qui est le type `list`, et font partie des quatre types natifs de Python qui permettent de stocker des collections de données, avec le `set` (similaires aux ensembles mathématiques : c'est une liste sans doublons et sans notion d'ordre entre les éléments), le `tuple` (assimilable aux n–uplets en mathématiques) et le `dict` pour dictionnaire, que vous verrez plus tard.

Créer des listes

Il existe deux principales façon d'instancier (de créer) des listes.

La première consiste à définir de façon exhaustive l'ensemble des éléments que contient notre liste. Pour ce faire, on utilise des crochets pour délimiter le début (`[`) et la fin (`]`) de notre liste. Ainsi, si l'on veut instancier une liste vide dans la variable `listeVide`, on écrira `listeVide = []`.

Si l'on veut instancier une liste avec un unique élément, on viendra préciser cet élément entre les crochets. Notez que l'on peut soit indiquer directement une valeur, ou bien choisir de préciser une variable que l'on aura déjà définie à l'avance.

Créer des listes

- `maListe = [4]`
- `maValeur = 4`
  `maListe = [maValeur]`

créeront tous deux une liste ne contenant qu'un seul élément qui vaut 4.

Pour créer des listes avec plusieurs éléments, on vient spécifier (dans l'ordre) les éléments entre les crochets en les séparant par une virgule (`,`).
La liste des 5 premiers entiers s'écrit donc
`cinqPremiersEntiers = [1, 2, 3, 4, 5]`.

Créer des listes

L'autre façon de créer des listes consiste à ne pas spécifier l'ensemble des éléments à son instanciation, mais à ajouter des éléments dans la liste au fur et à mesure.

Pour ce faire, on peut créer une liste vide, ou partir d'une liste déjà partiellement complétée en suivant la méthode décrite dans les slides précédentes. Puis, on peut utiliser la méthode `append()` sur cette liste afin d'ajouter un élément que l'on précise entre les parenthèses à la **fin de la liste**.

Ainsi, pour transformer notre liste des cinq premiers entiers en une liste des dix premiers entiers, on peut procéder comme suit :

Créer des listes

```
dixPremiersEntiers = cinqPremiersEntiers
dixPremiersEntiers.append(6)
dixPremiersEntiers.append(7)
dixPremiersEntiers.append(8)
dixPremiersEntiers.append(9)
dixPremiersEntiers.append(10)
```

Enfin, il est possible d'obtenir la taille d'une liste, à savoir, le nombre d'éléments qu'elle contient. Il suffit pour cela d'appliquer la fonction Python `len()` à la liste, en spécifiant le nom de la variable qui représente notre liste entre les parenthèse. Ainsi, `len(dixPremiersEntiers)` retournera 10 après l'exécution du code ci-dessus.

Accéder à un élément d'une liste

On considère ici maintenant avoir une liste Python qui est instanciée.

Un des gros avantages des listes en Python est que l'on peut accéder à ses éléments par sa position. Ce numéro de position attribué à chaque élément est appelé indice, **qui commence systématiquement à 0**.

Ainsi, si l'on reprend l'exemple de `dixPremiersEntiers`, la valeur 1 sera associée à l'indice 0, la valeur 2 à l'indice 1, et ainsi de suite jusqu'à la valeur 10 qui sera associée à l'indice 9.

De façon plus générale, une liste comportant n éléments sera indicée de 0 à $(n-1)$ inclus.

Accéder à un élément d'une liste

Finalement, pour accéder à l'élément associé à une position donnée, on écrit `maListe[indiceChoisi]`, où `maListe` est la liste considérée et `indiceChoisi` l'indice de cette liste dont on veut connaître la valeur associée.

Attention toutefois : comme on peut accéder à un élément directement en précisant son indice, il faut s'assurer que l'indice précisé est bien un indice valide pour la liste, à savoir un indice compris entre 0 et la taille de la liste moins un. Si vous indiquez un indice invalide, vous obtiendrez un message d'erreur du type "IndexError : list index out of range".

Accéder à un élément en utilisant l'indice permet aussi de modifier la valeur de la liste à cet indice : il suffit alors de faire `maListe[indiceChoisi] = nouvelleValeur`.

Accès par tranche et accès au dernier élément

On peut accéder au dernier élément d'une liste sans indiquer explicitement l'indice de ce dernier élément. Il suffit d'indiquer l'indice « -1 »entre crochets, comme suit : `maListe[-1]`. On peut continuer ainsi et obtenir l'avant-dernier élément (`maListe[-2]`), ...

Un dernier accès notable et pratique est le tranchage (appelé *slicing* en anglais). Il permet de sélectionner une sous-liste (une partie) d'une liste d'origine. Pour ce faire, il suffit d'indiquer dans les crochets un indiçage sur le modèle `[m:n+1]` pour obtenir la sous-liste qui contient tous les éléments entre les indices m et n inclus (écrit autrement, entre les indices m inclus et $n+1$ exclus). Enfin, pour recouvrir tous les éléments d'une liste, on utilise `[:]`.

Exemple : pour extraire la sous-liste de la liste d'origine `dixPremiersEntiers` contenant les entiers de 3 à 7 inclus, on écrira `dixPremiersEntiers[2:7]`.

Supprimer un élément d'une liste

Il est également possible de supprimer un élément d'une liste Python. Pour ce faire, on utilise la syntaxe suivante :

```
del maListe[indiceElementASupprimer]
```

La liste sera ensuite ré-indicée en prenant en compte la suppression. Ainsi, même si vous supprimez un élément au milieu de la liste, tous les éléments qui suivent auront leur indice actualisé pour toujours avoir des indices allant de 0 à la taille de la liste moins un. Bien entendu, la taille de la liste est aussi actualisée en conséquence.

Il est aussi possible de supprimer le dernier élément d'une liste en utilisant `maListe.pop()`.

Opérations sur les listes

Il existe deux opérateurs initialement arithmétiques qui peuvent être utilisés sur les listes :

- Le `+`, qui sert d'opérateur de concaténation. Il permet de mettre bout à bout deux (ou plus) liste les unes après les autres pour former une seule, grande, liste. Par exemple :
  `print([1, 2, 3] + [4, 5, 6, 7])` donnera
  `[1, 2, 3, 4, 5, 6, 7]`.
- Le `*`, qui sert d'opérateur de duplication. Il permet de répéter un nombre de fois donné une liste. Par exemple :
  `print([8, 4, 2] * 4)` donnera
  `[8, 4, 2, 8, 4, 2, 8, 4, 2, 8, 4, 2]`.

A vous de jouer !

Exercice

Créer une liste Python `semaine` *contenant les sept jours de la semaine. A partir de cette liste, récupérer deux sous-listes* `joursTravail` *et* `weekend` *contenant respectivement les jours ouvrés de la semaine et les jours de weekend, et ce de deux façons différentes. Proposer ensuite deux manières d'accéder au dernier jour de la semaine.*